

JAVA OOP PRACTICAL RECORD

Library Management System

Experiment No.	02
Date	29/08/2026
Title	Library Management System – OOP Concepts

1. Objective / Aim

To write a Java program that demonstrates fundamental Object-Oriented Programming concepts through a Library Management System, including classes and objects, constructors, encapsulation, inheritance, polymorphism, abstraction, and interfaces.

2. System Requirements

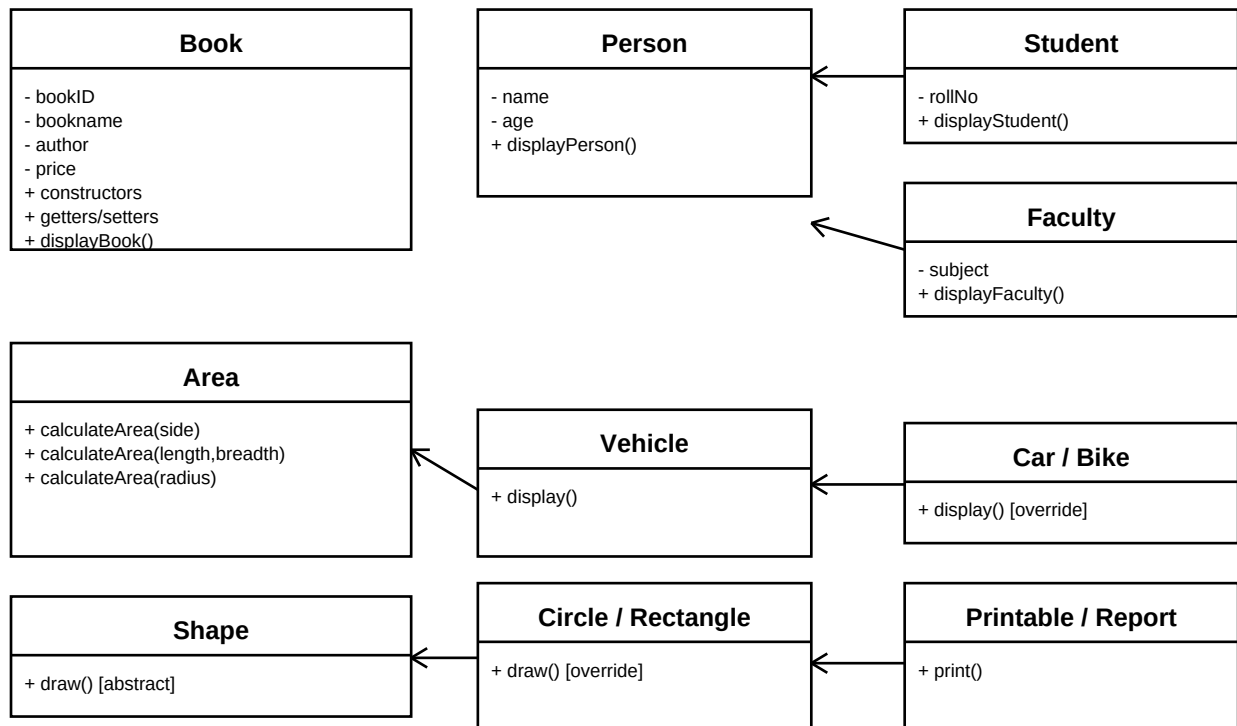
Hardware: Standard PC/Laptop with 4 GB/8 GB RAM and a Core i3/i5 processor or equivalent.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, and a Java editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

3. Algorithm / Logic Description

1. Create the Book class with private data members and provide default and parameterized constructors.
2. Implement getters and setters to demonstrate encapsulation and create Book objects.
3. Create Person as a base class and derive Student and Faculty using inheritance.
4. Create Area with overloaded calculateArea() methods for square, rectangle, and circle.
5. Create Vehicle and override display() in Car and Bike to demonstrate runtime polymorphism.
6. Create abstract Shape and implement draw() in Circle and Rectangle.
7. Create Printable interface and implement it in Report.
8. Create objects in main() and invoke all methods to display the results.

4. UML Class Diagram



5. Source Code

```

package observation.02;

// PART A and B: Classes, Objects, Constructors, Encapsulation
class Book {
    private int bookID;
    private String bookname;
    private String author;
    private double price;

    // Default constructor
    Book() {
        bookID = 0;
        bookname = "Unknown";
        author = "Unknown";
        price = 0.0;
    }

    // Parameterized constructor
    Book(int bookID, String bookname, String author, double price) {
        this.bookID = bookID;
        this.bookname = bookname;
        this.author = author;
        this.price = price;
    }

    // Getters and Setters
    int getbookID() { return bookID; }
    void setbookID(int bookID) { this.bookID = bookID; }

    String getbookname() { return bookname; }
    void setbookname(String bookname) { this.bookname = bookname; }

    String getauthor() { return author; }
    void setauthor(String author) { this.author = author; }

    double getprice() { return price; }
    void setprice(double price) { this.price = price; }

    // Display book details
    void displayBook() {
        System.out.println("Book ID : " + bookID);
        System.out.println("Book Name : " + bookname);
        System.out.println("Author : " + author);
        System.out.println("Price : " + price);
    }
}

// PART C: INHERITANCE
class Person {
    String name;
    int age;

```

```

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void displayPerson() {
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
    }
}

// Student inherits Person
class Student extends Person {
    int rollNo;

    Student(String name, int age, int rollNo) {
        super(name, age);
        this.rollNo = rollNo;
    }

    void displayStudent() {
        displayPerson();
        System.out.println("Roll No : " + rollNo);
    }
}

// Faculty inherits Person
class Faculty extends Person {
    String subject;

    Faculty(String name, int age, String subject) {
        super(name, age);
        this.subject = subject;
    }

    void displayFaculty() {
        displayPerson();
        System.out.println("Subject : " + subject);
    }
}

// PART D: POLYMORPHISM
// Method Overloading
class Area {
    double calculateArea(double side) {
        return side * side;
    }

    double calculateArea(double length, double breadth) {
        return length * breadth;
    }

    double calculateArea(float radius) {
        return 3.14 * radius * radius;
    }
}

// Method Overriding
class Vehicle {
    void display() {
        System.out.println("This is a Vehicle");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Car");
    }
}

class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Bike");
    }
}

// PART E: ABSTRACTION
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {

```

```

        System.out.println("Drawing a Rectangle");
    }
}

// INTERFACE
interface Printable {
    void print();
}

class Report implements Printable {
    @Override
    public void print() {
        System.out.println("Printing Library Report");
    }
}

public class LibraryManagementSystem {
    public static void main(String[] args) {

        // PART A + B: Book Objects
        System.out.println("==== BOOK DETAILS =====");

        Book b1 = new Book(101, "Java Programming", "Santhosh", 500);
        b1.displayBook();
        System.out.println();

        Book b2 = new Book();

        b2.setbookID(102);
        b2.setbookname("Data Structures");
        b2.setauthor("Ganga Parvathi");
        b2.setprice(450);

        b2.displayBook();

        System.out.println("\nBook Name using Getter: " + b1.getbookname());

        // PART C: Inheritance
        System.out.println("\n===== INHERITANCE =====");

        Student s = new Student("Avinash", 20, 106);
        s.displayStudent();

        System.out.println();

        Faculty f = new Faculty("Dr.James", 40, "Computer Science");
        f.displayFaculty();

        // PART D: Method Overloading
        System.out.println("\n===== METHOD OVERLOADING =====");

        Area a = new Area();

        System.out.println("Area of Square: " + a.calculateArea(5.0));
        System.out.println("Area of Rectangle: " + a.calculateArea(5.0, 4.0));
        System.out.println("Area of Circle: " + a.calculateArea(3.0f));

        // PART D: Method Overriding
        System.out.println("\n===== METHOD OVERRIDING =====");

        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();

        v1.display();
        v2.display();

        // PART E: Abstraction
        System.out.println("\n===== ABSTRACTION =====");

        Shape c = new Circle();
        Shape r = new Rectangle();

        c.draw();
        r.draw();

        // PART E: Interface
        System.out.println("\n===== INTERFACE =====");

        Report report = new Report();
        report.print();
    }
}

```

6. Compilation & Execution Commands

Because the program contains a package declaration, save the file as **LibraryManagementSystem.java** inside a folder named **observation**. Note: Java package names normally should not contain a dot as a folder name; use the package structure required by your IDE/environment.

```
javac -d . LibraryManagementSystem.java
java observation.02.LibraryManagementSystem
```

7. Sample Output Screen

```
===== BOOK DETAILS =====
Book ID   : 101
Book Name : Java Programming
Author    : Santhosh
Price     : 500.0

Book ID   : 102
Book Name : Data Structures
Author    : Ganga Parvathi
Price     : 450.0

Book Name using Getter: Java Programming

===== INHERITANCE =====
Name : Avinash
Age  : 20
Roll No : 106

Name : Dr.James
Age  : 40
Subject : Computer Science

===== METHOD OVERLOADING =====
Area of Square: 25.0
Area of Rectangle: 20.0
Area of Circle: 28.26

===== METHOD OVERRIDING =====
This is a Car
This is a Bike

===== ABSTRACTION =====
Drawing a Circle
Drawing a Rectangle

===== INTERFACE =====
Printing Library Report
```

8. Result

The Library Management System was successfully implemented. The program demonstrates classes and objects, constructors, encapsulation through private members and getters/setters, inheritance, method overloading, method overriding, abstraction, and interface implementation.